

1)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/inotify.h>
#include <pthread.h>
#include <limits.h>

/* Tamanhos de buffer para leitura do inotify */
#define NAME_MAX 255
#define EVENT_SIZE (sizeof(struct inotify_event))
#define BUF_LEN (1024 * (EVENT_SIZE + NAME_MAX + 1))

/* Estrutura para representar uma tarefa na fila */
typedef struct Task {
    int type; // Máscara do evento inotify
    char name[NAME_MAX]; // Nome do arquivo/diretório
    struct Task *next;
} Task;

/* Variáveis Globais (Compartilhadas entre threads) */
char *SRC_DIR;
char *DST_DIR;

Task *queue_head = NULL;
Task *queue_tail = NULL;

pthread_mutex_t queue_mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t queue_cond = PTHREAD_COND_INITIALIZER;

/* --- Funções da Fila --- */

void enqueue(int type, const char *name) {
    Task *new_task = (Task *)malloc(sizeof(Task));
    new_task->type = type;
    strncpy(new_task->name, name, NAME_MAX);
    new_task->next = NULL;

    if (queue_tail == NULL) {
        queue_head = new_task;
        queue_tail = new_task;
    } else {
        queue_tail->next = new_task;
        queue_tail = new_task;
    }
}

Task *dequeue() {
```

```

    if (queue_head == NULL) return NULL;

    Task *tmp = queue_head;
    queue_head = queue_head->next;
    if (queue_head == NULL) queue_tail = NULL;

    return tmp;
}

/* --- Função Auxiliar para executar comandos de sistema --- */
void sync_action(int type, const char *filename) {
    char cmd[4096];
    char src_path[2048];
    char dst_path[2048];

    // Monta caminhos completos
    snprintf(src_path, sizeof(src_path), "%s/%s", SRC_DIR, filename);
    snprintf(dst_path, sizeof(dst_path), "%s/%s", DST_DIR, filename);

    // Ignora arquivos ocultos ou temporários do próprio editor se
    necessário
    if (filename[0] == '.') return;

    printf("[WORKER] Processando arquivo: %s (Mask: %d)\n", filename,
type);

    if (type & IN_CREATE || type & IN_MOVED_TO) {
        // Se for diretório, cria diretório, senão copia arquivo
        if (type & IN_ISDIR) {
            snprintf(cmd, sizeof(cmd), "mkdir -p \"%s\"", dst_path);
        } else {
            // cp -r cobre arquivos e diretórios recursivamente se
            necessário
            snprintf(cmd, sizeof(cmd), "cp -r \"%s\" \"%s\"", src_path,
dst_path);
        }
        system(cmd);
    }
    else if (type & IN_CLOSE_WRITE) {
        // Arquivo terminou de ser escrito, copia atualização
        snprintf(cmd, sizeof(cmd), "cp -r \"%s\" \"%s\"", src_path,
dst_path);
        system(cmd);
    }
    else if (type & IN_DELETE || type & IN_MOVED_FROM) {
        // Remove no destino
        snprintf(cmd, sizeof(cmd), "rm -rf \"%s\"", dst_path);
        system(cmd);
    }
}

/* --- Thread Consumidora --- */
void *worker_thread(void *arg) {
    while (1) {

```

```

    Task *task = NULL;

    // Região Crítica: Acesso à Fila
    pthread_mutex_lock(&queue_mutex);

    // Enquanto fila vazia, dorme e espera sinal
    while (queue_head == NULL) {
        pthread_cond_wait(&queue_cond, &queue_mutex);
    }

    task = dequeue();

    pthread_mutex_unlock(&queue_mutex);
    // Fim da Região Crítica

    if (task != NULL) {
        sync_action(task->type, task->name);
        free(task);
    }
}
return NULL;
}

/* --- Main (Thread Produtora) --- */
int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "Uso: %s <diretorio_origem> <diretorio_destino>\n",
argv[0]);
        exit(1);
    }

    SRC_DIR = argv[1];
    DST_DIR = argv[2];

    int fd, wd;
    char buffer[BUF_LEN];

    // 1. Inicializa Inotify
    fd = inotify_init();
    if (fd < 0) {
        perror("inotify_init");
        exit(1);
    }

    // 2. Adiciona Monitoramento
    // Monitoramos: Criação, Deleção, Modificação (close_write), Mover
    wd = inotify_add_watch(fd, SRC_DIR,
        IN_CREATE | IN_DELETE | IN_CLOSE_WRITE | IN_MOVED_FROM |
IN_MOVED_TO);

    if (wd < 0) {
        perror("inotify_add_watch");
        printf("Certifique-se que o diretorio '%s' existe.\n", SRC_DIR);
        exit(1);
    }
}

```

```

}

printf("Monitorando '%s' -> Espelhando em '%s'\n", SRC_DIR, DST_DIR);

// 3. Cria a Thread Trabalhadora
pthread_t tid;
if (pthread_create(&tid, NULL, worker_thread, NULL) != 0) {
    perror("pthread_create");
    exit(1);
}

// 4. Loop Principal (Produtor)
while (1) {
    int length, i = 0;

    // Leitura bloqueante - o programa para aqui até algo acontecer no
    // diretório
    length = read(fd, buffer, BUF_LEN);

    if (length < 0) {
        perror("read");
        break;
    }

    while (i < length) {
        struct inotify_event *event = (struct inotify_event *)
        &buffer[i];

        if (event->len) {
            // Adiciona evento na fila de forma segura
            pthread_mutex_lock(&queue_mutex);
            enqueue(event->mask, event->name);

            // Avisa a thread trabalhadora (sinaliza)
            pthread_cond_signal(&queue_cond);
            pthread_mutex_unlock(&queue_mutex);
        }

        i += EVENT_SIZE + event->len;
    }
}

inotify_rm_watch(fd, wd);
close(fd);
return 0;
}

```

2)

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <string.h>
#include <dirent.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>
#include <pwd.h>
#include <grp.h>
#include <time.h>

// Função auxiliar para imprimir as permissões (ex: rwxr-xr-x)
void print_permissions(mode_t mode) {
    printf( (S_ISDIR(mode)) ? "d" : "-");
    printf( (mode & S_IRUSR) ? "r" : "-");
    printf( (mode & S_IWUSR) ? "w" : "-");
    printf( (mode & S_IXUSR) ? "x" : "-");
    printf( (mode & S_IRGRP) ? "r" : "-");
    printf( (mode & S_IWGRP) ? "w" : "-");
    printf( (mode & S_IXGRP) ? "x" : "-");
    printf( (mode & S_IROTH) ? "r" : "-");
    printf( (mode & S_IWOTH) ? "w" : "-");
    printf( (mode & S_IXOTH) ? "x" : "-");
}

int main(int argc, char *argv[]) {
    // Verifica se o diretório foi passado
    if (argc < 2) {
        fprintf(stderr, "Uso: %s <caminho_do_diretorio>\n", argv[0]);
        exit(1);
    }

    const char *dir_path = argv[1];
    struct dirent *entry;
    struct stat file_stat;
    DIR *dp;

    // Buffer para armazenar o caminho completo (diretorio + / +
    nome_arquivo)
    char full_path[4096];

    // Abre o diretório
    dp = opendir(dir_path);
    if (dp == NULL) {
        perror("Erro ao abrir diretorio");
        exit(1);
    }

    printf("Lendo propriedades do diretório: %s\n\n", dir_path);
    printf("%-11s %-10s %-10s %-10s %-20s %s\n",
        "PERMISSOES", "USUARIO", "GRUPO", "TAMANHO", "MODIFICADO EM",
        "NOME");
    printf("-----\n");

    // Loop para ler cada arquivo dentro do diretório

```

```

while ((entry = readdir(dp))) {
    // Pula os diretórios "." (atual) e ".." (pai)
    if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name, "..")
== 0)
        continue;

    // Monta o caminho completo para que o stat() funcione
    // Ex: se dir é "/tmp" e arquivo é "a.txt", full_path vira
"/tmp/a.txt"
    snprintf(full_path, sizeof(full_path), "%s/%s", dir_path, entry-
>d_name);

    // Obtém os metadados do arquivo
    if (stat(full_path, &file_stat) == -1) {
        perror("stat");
        continue;
    }

    // 1. Imprime Permissões
    print_permissions(file_stat.st_mode);
    printf(" ");

    // 2. Obtém Nome do Usuário (Dono)
    struct passwd *pw = getpwuid(file_stat.st_uid);
    if (pw) printf("%-10s ", pw->pw_name);
    else    printf("%-10d ", file_stat.st_uid); // Se não achar nome,
imprime ID

    // 3. Obtém Nome do Grupo
    struct group *gr = getgrgid(file_stat.st_gid);
    if (gr) printf("%-10s ", gr->gr_name);
    else    printf("%-10d ", file_stat.st_gid);

    // 4. Tamanho do arquivo (bytes)
    printf("%-10ld ", file_stat.st_size);

    // 5. Data de Modificação
    char date_str[20];
    struct tm *tm_info = localtime(&file_stat.st_mtime);
    strftime(date_str, sizeof(date_str), "%Y-%m-%d %H:%M", tm_info);
    printf("%-20s ", date_str);

    // 6. Nome do Arquivo
    printf("%s\n", entry->d_name);
}

closedir(dp);
return 0;
}

```